AIORI 3
HACKATHON

A3-PS034-TC018

Schema-Validated YANG-Push Telemetry Pipeline

PS-034 | Cloud Computing & IoT

Team: Beyond Limits
Team Lead: Alagammal V EEE
Team Member: Sherlin CSE
Team size: 2
Mentor : Dr. Saranya karuppasamy EEE/AP

Abstract & Problem Description

ABSTRACT

A telemetry pipeline receives operational data from network devices. This prototype validates simulated JSON notifications, separates valid and invalid records, records rejection reasons, and presents the results in a Streamlit dashboard.

PROBLEM DESCRIPTION

Malformed, incomplete, or out-of-range telemetry can reduce monitoring reliability. If invalid data reaches downstream monitoring, it can create misleading metrics and make troubleshooting more difficult.

Key issues being addressed

Required fields may be missing or have incorrect data types.

Numeric measurements may fall outside acceptable ranges.

Rejected notifications need clear reasons for diagnosis.

Operators need an at-a-glance view of accepted and rejected data.

Background

TELEMETRY

Operational measurements include CPU usage, memory usage, bandwidth, packet loss, and latency.

YANG

A data-modeling language used to describe structured network configuration and operational data.

YANG-PUSH

A subscription mechanism that sends selected datastore updates to a receiver.

Background to the proposed optimization

A validation stage can identify malformed or incomplete payloads before they are treated as usable data.

File-based logs make accepted records and rejection reasons easy to inspect in the prototype.

A dashboard gives a consolidated view of processing results and supports manual test cases.

The current model is a Python simulation; real YANG schema and live YANG-Push integration are planned next stages.

Literature & RFC Review

RFC 7950 — YANG 1.1

Defines the YANG data-modeling language.

RFC 8639 — Subscription to YANG Notifications

Defines a subscription framework for event notifications and streamed updates.

RFC 8641 — YANG-Push

Describes subscription-based delivery of datastore updates.

RFC 8259 — JSON

Defines the JSON format used by the prototype's simulated notifications.

Relevance to the proposed system

These standards provide context for the planned data model, notification subscriptions, datastore updates, and JSON representation. The current implementation demonstrates rule-based validation, not full standards conformance.

Proposed Solution

1 **RECEIVE**

Simulated JSON notification

2 **VALIDATE**

Required fields, types and ranges

3 **CLASSIFY**

Accepted or rejected

4 **RECORD**

Local log files

5 **DISPLAY**

Dashboard metrics, tables and chart

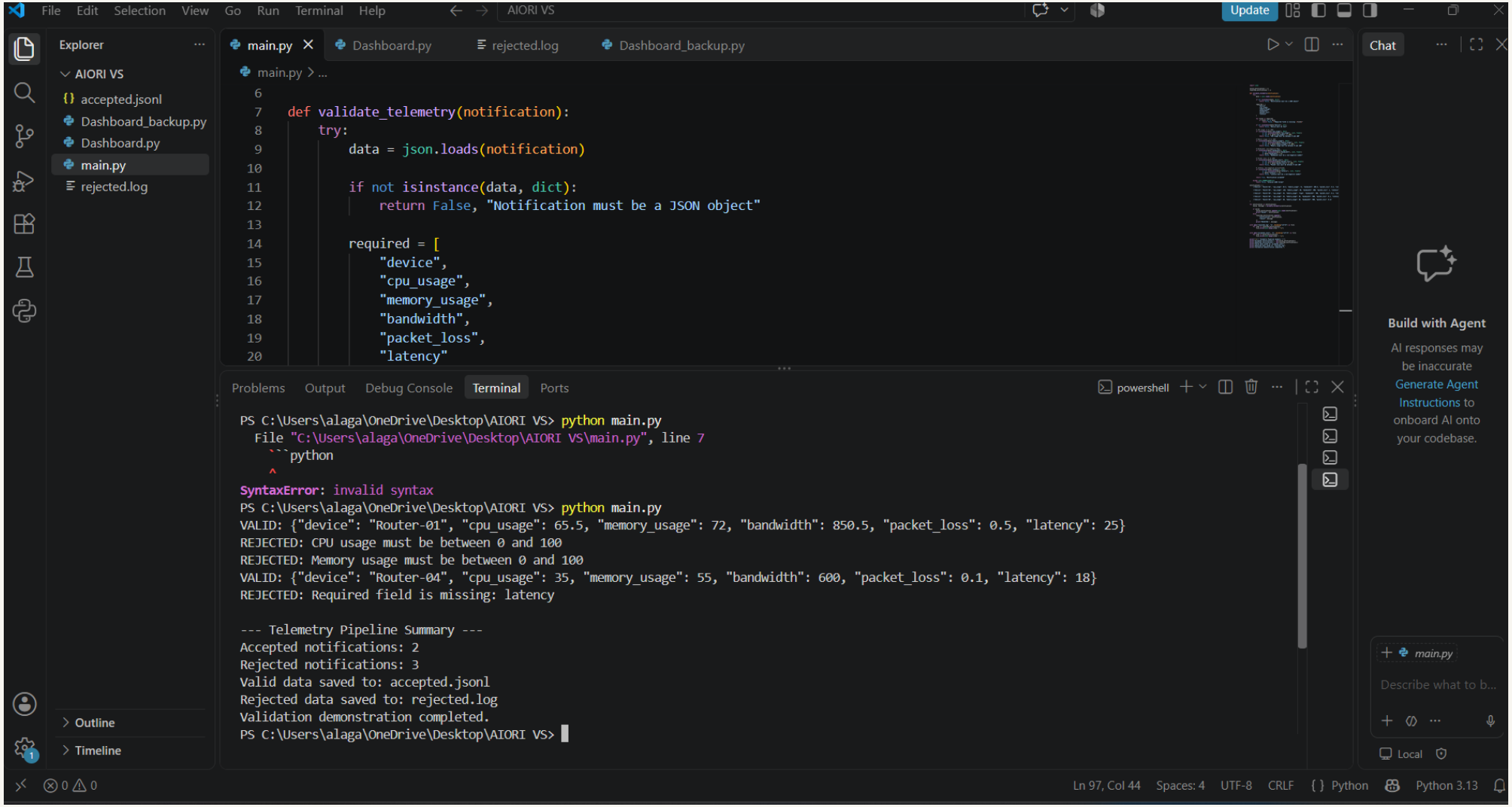
Telemetry fields validated in the current model

device • cpu_usage (0–100) • memory_usage (0–100) • bandwidth (≥0) • packet_loss (0–100) • latency (≥0)

Full YANG-schema validation and message-broker integration are future development stages.

Basic Model Evidence — Screenshot 1

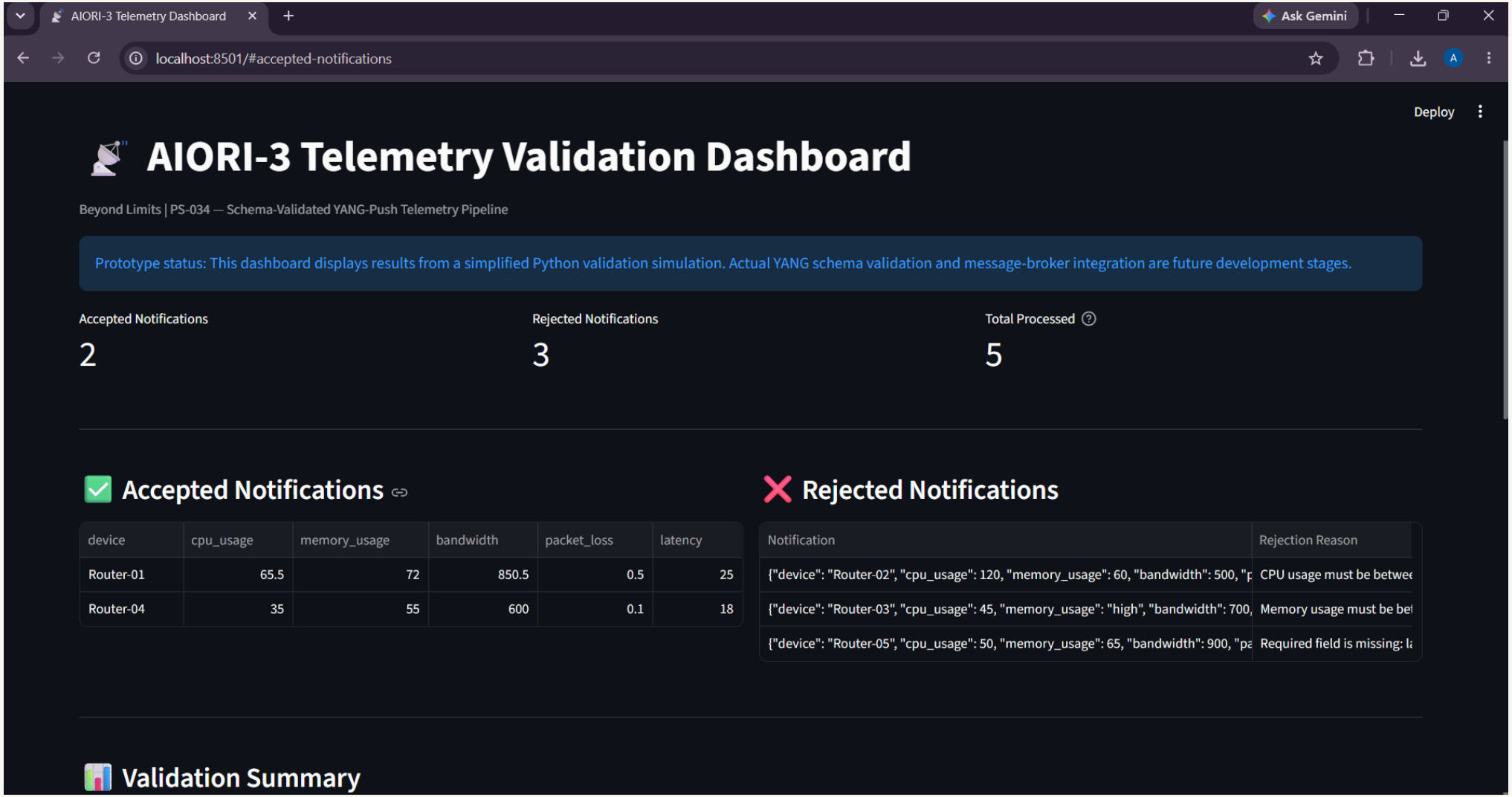
VS Code: main.py and terminal validation output



Evidence from the working Python simulation and Streamlit dashboard.

Basic Model Evidence — Screenshot 2

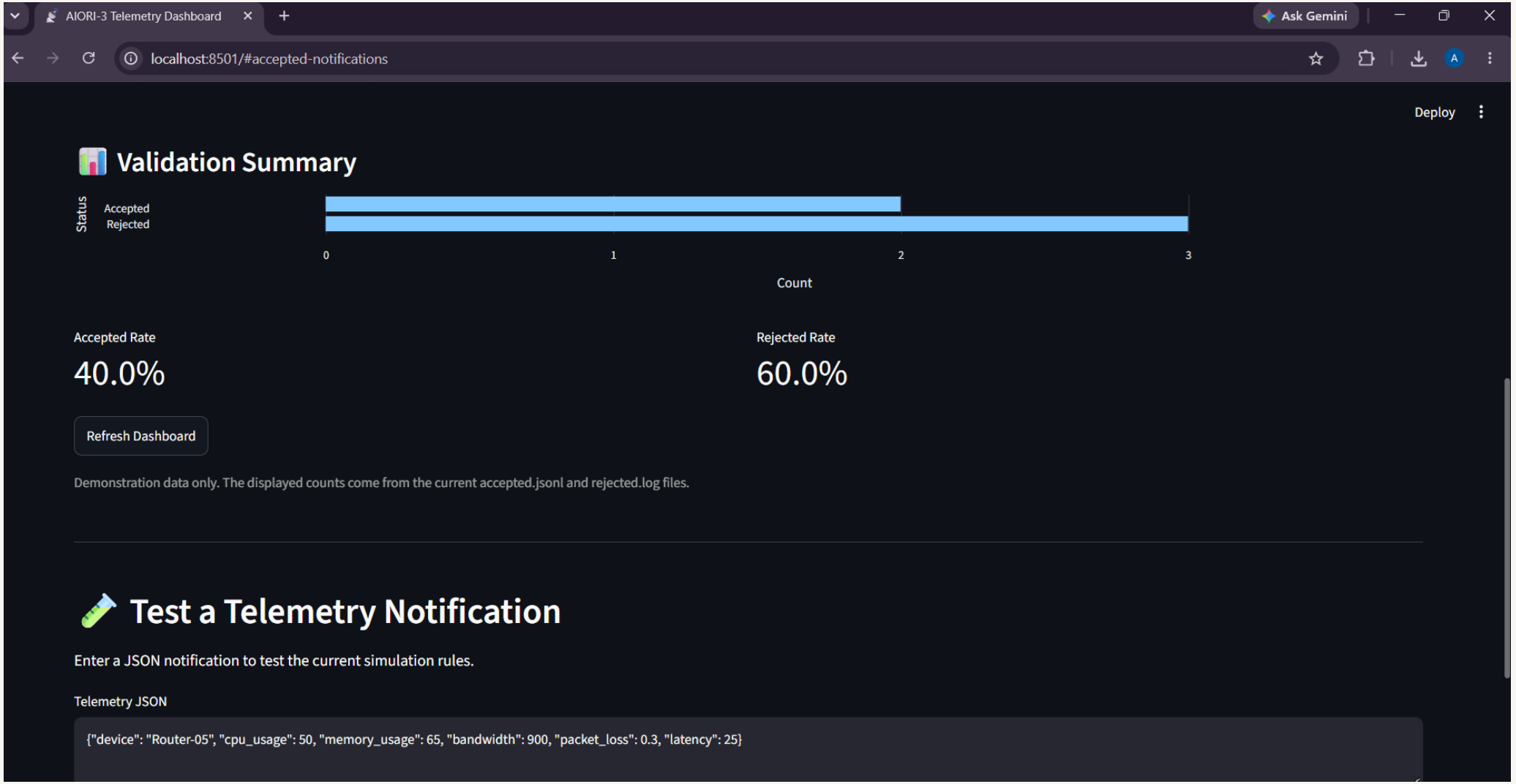
Dashboard overview: counts and accepted/rejected records



Evidence from the working Python simulation and Streamlit dashboard.

Basic Model Evidence — Screenshot 3

Validation summary: chart and acceptance/rejection rates



Evidence from the working Python simulation and Streamlit dashboard.

Basic Model Evidence — Screenshot 4

Valid test case: notification accepted



Evidence from the working Python simulation and Streamlit dashboard.

Basic Model Evidence — Screenshot 5

Invalid test case: cpu_usage = 120 rejected



Evidence from the working Python simulation and Streamlit dashboard.

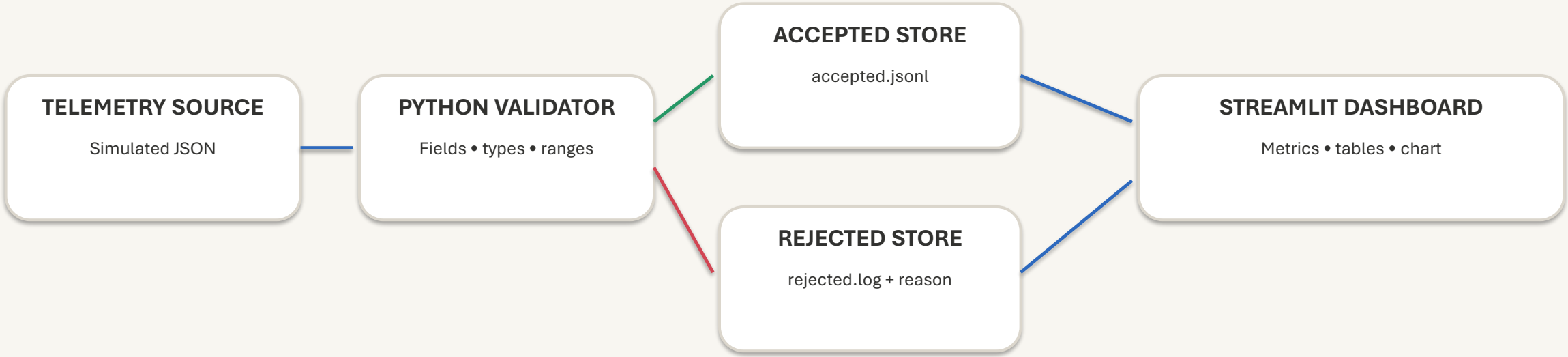
Solution Optimization

Aspect	Basic / before	Current prototype / after
Validation	No consistent validation step	Checks required fields, data types, and ranges
Error handling	Invalid payload can be hard to diagnose	Rejected records include a reason
Traceability	Manual inspection of terminal output	Accepted and rejected results saved to files
Visibility	No consolidated view	Dashboard metrics, tables, and chart

Demonstration result: 5 sample notifications → 2 accepted and 3 rejected.

Counts reflect the sample dataset, not a performance benchmark.

Solution Architecture & Design (UML Diagram)



Basic model evidence

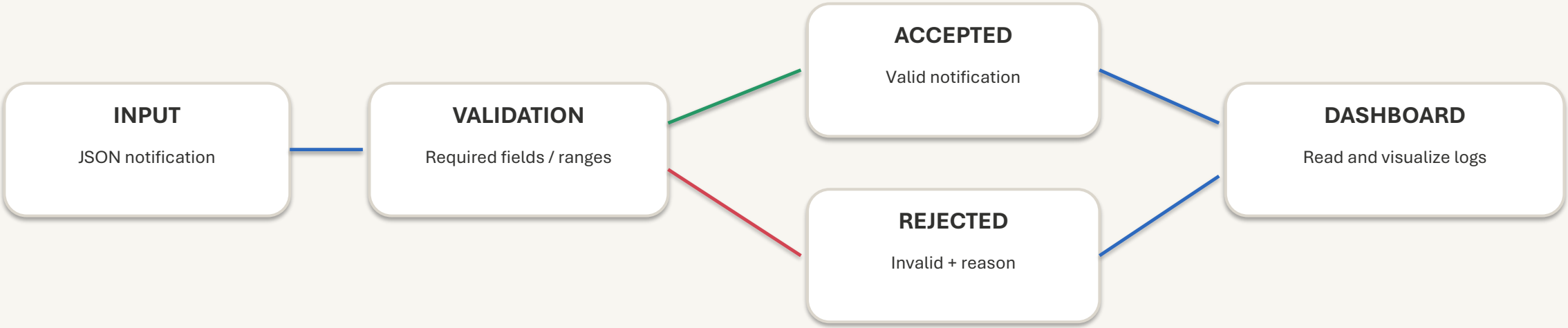
VS Code: main.py runs the validation simulation.

Terminal: 2 accepted and 3 rejected sample notifications.

Dashboard: totals, accepted/rejected tables, validation chart, and interactive test section.

Figure: System Architecture UML Diagram (logical prototype architecture)

Data Flow Diagram (DFD)



DATA STORE: accepted.jsonl

Stores one JSON record per accepted notification.

DATA STORE: rejected.log

Stores rejected notifications and validation reasons.

Figure: Data Flow Diagram (DFD) — current simulated data flow.

Deployment Timeline

1

Prototype

Python + JSON

2

Validation

Fields, types, ranges

3

Dashboard

Metrics + tests

4

YANG validation

Add real schema validation

5

Live pipeline

YANG-Push + collector

Deployment methodology

Test valid and invalid notifications in each stage.

Verify saved logs and dashboard outputs.

Integrate real standards-based components after the simulation is stable.

Perform reliability and error-handling tests before deployment.

Bibliography

[1] M. Björklund, “The YANG 1.1 Data Modeling Language,” RFC 7950, 2016. https://www.rfc-editor.org/rfc/rfc7950

[2] E. Voit et al., “Subscription to YANG Notifications,” RFC 8639, 2019. https://www.rfc-editor.org/rfc/rfc8639

[3] E. Voit et al., “Subscription to YANG Notifications for Datastore Updates,” RFC 8641, 2019. https://www.rfc-editor.org/rfc/rfc8641

[4] T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format,” RFC 8259, 2017. https://www.rfc-editor.org/rfc/rfc8259

[5] Streamlit Documentation. https://docs.streamlit.io/

[6] Python JSON module. https://docs.python.org/3/library/json.html

References inform the technical background; the current prototype does not claim full RFC conformance.

THANK YOU FOR YOUR ATTENTION!

Schema-Validated YANG-Push Telemetry Pipeline

AIORI-3 • PS-034 • Team Beyond Limits

Questions?